

Reviewer Pack — Real Rank of Karchmer-Wigderson Communication Matrices for A...

Entry c26787a56682 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-30 17:18:30 UTC

Real Rank of Karchmer-Wigderson Communication Matrices for AC⁰ PARITY

Entry ID: c26787a56682 Verdict: INCONCLUSIVE Recorded: 2026-04-30 17:18:30 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): Karchmer-Wigderson Communication Complexity

Statement:

For any AC⁰ circuit C computing PARITY on n inputs, the real rank of its Karchmer-Wigderson communication matrix M_C satisfies $\text{rank}_R(M_C) \geq \Omega(\sqrt{n})$.

Rationale (proposer’s reasoning):

The real rank of the communication matrix captures geometric constraints on the function’s representation. For PARITY, which is linear, the matrix’s real rank may inherently resist AC⁰ approximations due to the function’s algebraic simplicity, offering a novel angle to AC⁰ lower bounds.

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dc59e49cdb545b65

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: INCONCLUSIVE against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Karchmer-Wigderson real rank communication matrix AC⁰ PARITY - Real Algebraic Geometry Karchmer-Wigderson communication matrix PARITY - lower bound real rank Karchmer-Wigderson AC⁰ PARITY communication matrix

Top relevant hits considered: - [http://arxiv.org/abs/2107.05128v2] Karchmer-Wigderson Games for Hazard-free Computation - [http://arxiv.org/abs/2002.07444v1] Multiparty Karchmer-Wigderson Games and Threshold Circuits - [http://arxiv.org/abs/2211.06357v2] Parity of ranks of Jacobians of curves - [http://arxiv.org/abs/1409.1534v1] Algorithms in Real Algebraic Geometry: A Survey - [http://arxiv.org/abs/1912.00347v3] Equiresidual algebraic geometry I: The affine theory - [http://arxiv.org/abs/1806.02734v3] Spectral lower bounds for the orthogonal and projective ranks of a graph

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=123.3s

5.1 Generated Python source

```
import random
import math
from itertools import product

def gaussian_elimination(matrix):
    n = len(matrix)
    augmented_matrix = [row[:] + [1 if i == j else 0 for j in range(n)] for i, row in enumerate(matrix)]

    for i in range(n):
        # Find the pivot row
        max_row = i
        for j in range(i+1, n):
            if abs(augmented_matrix[j][i]) > abs(augmented_matrix[max_row][i]):
                max_row = j

        # Swap rows
        augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]

        # Eliminate below the pivot
        for j in range(i+1, n):
            factor = augmented_matrix[j][i] / augmented_matrix[i][i]
            for k in range(n + 1):
                augmented_matrix[j][k] -= factor * augmented_matrix[i][k]

    return [row[-1] for row in augmented_matrix]

def compute_real_rank(matrix):
    try:
        rank = len(gaussian_elimination(matrix))
```

```

except ZeroDivisionError:
    rank = 0
return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 10
    c = 0.5

    # Generate a random AC0 circuit computing PARITY on n inputs
    def parity(x):
        return sum(int(bit) for bit in x) % 2

    M_C = [[parity(bin(i)[2:].zfill(n)) ^ parity(bin(j)[2:].zfill(n)) for j in range(1 << n)] for i in range(1 << n)]

    rank = compute_real_rank(M_C)

    return {
        "metric_name": "real_rank",
        "metric_value": rank,
        "instances_tested": 1,
        "conjecture_holds": rank >= c * math.sqrt(n),
        "counterexample": "" if rank >= c * math.sqrt(n) else "rank < c√n"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value)**2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] for res in results) and support_fraction >= 0.8:
        first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"rank < c√n\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

s': False, 'counterexample': 'rank < c√n'
TRIAL: {'metric_name': 'real_rank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'rank < c√n'}
TRIAL: {'metric_name': 'real_rank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'rank < c√n'}

```


3. The full LLM transcripts are in `research/audit/c26787a56682.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/c26787a56682.tar.gz` (if generated)